

Cleaning Addresses in PostgreSQL

<https://gemini.google.com/app/fa23661b9dce4b79>

User prompt: can i clean addresses in postgres. Eg: addresses with duplicates but text slightly different

Response: Yes, absolutely. Cleaning semi-duplicate text addresses in PostgreSQL is a classic problem, and Postgres has some of the best built-in tools to handle it.

Since standard `DISTINCT` or `GROUP BY` won't catch addresses like "123 Main St." vs "123 Main Street, Apt 4", you need **fuzzy text matching** combined with **standardization**.

Here is how to approach it, from quick-and-dirty matches to robust cleaning pipelines.

Step 1: Standardize the Text (The Baseline)

Before doing any complex comparison, clean up casing, punctuation, and common abbreviations. Create a simple normalization function or run an update to unify common terms:

```
-- Create a helper function to standardize common address tokens
CREATE OR REPLACE FUNCTION normalize_address(raw_addr TEXT)
RETURNS TEXT AS $$
BEGIN
    RETURN lower(
        regexp_replace(
            regexp_replace(
                regexp_replace(raw_addr, '[^\w\s]', '', 'g'), -- Remove punctuation
                '\y(st|street)\y', 'st', 'g'),                -- Unify Street
                '\y(ave|avenue)\y', 'ave', 'g')              -- Unify Avenue
    );
END;
$$ LANGUAGE plpgsql IMMUTABLE;
```

Step 2: Enable Similarity Extensions

PostgreSQL provides two essential extensions for finding near-duplicate text: `pg_trgm` (trigram matching) and `fuzzystrmatch` (Levenshtein distance & Soundex).

Run this once in your database:

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;  
CREATE EXTENSION IF NOT EXISTS fuzzystrmatch;
```

Step 3: Find Near-Duplicates

Option A: Trigram Similarity (`pg_trgm`) — *Best Overall*

Trigrams break strings into 3-character chunks and compare overlap. It is fast, handles typos, and works great for address strings.

```
-- Set the similarity threshold (0.0 to 1.0, default is 0.3)  
SET pg_trgm.similarity_threshold = 0.6;  
  
-- Find pairs of addresses that are very similar  
SELECT  
    a.id AS id_1, a.address AS address_1,  
    b.id AS id_2, b.address AS address_2,  
    similarity(a.address, b.address) AS score  
FROM customer_addresses a  
JOIN customer_addresses b  
    ON a.id < b.id  
    AND a.address % b.address -- '%' means 'similar above threshold'  
ORDER BY score DESC;
```

Option B: Levenshtein Distance (`fuzzystrmatch`) — *Best for Small Typos*

Levenshtein measures the exact number of edit operations (insertions, deletions, substitutions) needed to turn string A into string B.

```
SELECT  
    a.address,  
    b.address,  
    levenshtein(lower(a.address), lower(b.address)) AS edit_distance
```

```
FROM customer_addresses a
JOIN customer_addresses b
  ON a.id < b.id
WHERE levenshtein(lower(a.address), lower(b.address)) BETWEEN 1 AND 3;
```

Step 4: De-duplicate using Window Functions

Once you identify duplicates, you can automatically group them and select a "canonical" (primary) record to keep—usually the longest string or the most recently updated one.

```
WITH RankedAddresses AS (
  SELECT
    id,
    address,
    -- Group addresses that match on normalized trigrams or exact street numbers
    FIRST_VALUE(id) OVER (
      PARTITION BY regexp_replace(address, '\D', '', 'g') -- Group by house/zip numbers
      ORDER BY length(address) DESC, id ASC              -- Keep longest address
    ) AS canonical_id
  FROM customer_addresses
)
SELECT * FROM RankedAddresses;
```

Pro-Tips for Scaling Large Datasets

1. **Add a GiST or GIN Index:** Fuzzy matching across millions of rows without an index will result in full-table scans.

```
CREATE INDEX idx_address_trgm ON customer_addresses USING gin (address gin_trgm_ops);
```

2. **Block by Zip Code or City First:** Never compare every address in the table against every other address ($O(N^2)$). Always filter down comparisons by matching Zip/Postal Code or City first:

```
JOIN customer_addresses b
  ON a.zip_code = b.zip_code
 AND a.id < b.id
 AND a.address % b.address
```

